

Assignment 4 - Project Proposal:

Title: Easy Trip: US Trip Planner

One sentence description: This is an AI powered US trip planner deployed as a hosted, public facing web app with user accounts, per user trip storage, and API cost controls (Ship with auth + live URL).

Past project reference: This project extends my own work started in late March 2025. I worked on it a bit during the end of spring break but stopped once school started back up. It's a new project (not directly based on A1 or A2 or A3). It's built independently as a personal side project.

- **GitHub:** <https://github.com/JustinC333/Easy-Trip>

Planned technologies:

- Framework: Next.js 16 (App Router), TypeScript
- Styling: Tailwind CSS v4
- Auth & DB: Supabase (Auth + PostgreSQL)
- AI: Anthropic Claude API (claude-sonnet)
- External APIs: Google Places API, National Park Service (NPS) API, RapidAPI (hotels/flights)
- Deployment: Vercel

First deliverable: The first thing I'll build would probably be a use story. A logged in user would fill out the multi step plan form, submit it, and receive a structured AI generated day by day itinerary saved to their account that they can look at later.

This will touch everything: auth, the multi step form, Claude for generation, NPS + Google Places for real data, Supabase to save it, and a results page to show it.

Rough Architecture for First Deliverable:

- AuthGate (Supabase Auth)
 - Input: email + password
 - Output: active session, JWT in cookies
 - Effect: creates a user row on sign-up; locks down /plan and /trips routes via Next.js middleware so randos can't hit them
- PlanForm (multi-step UI)
 - Input: user filling out 4 steps: destination, dates, group size, interests
 - Output: a TripRequest object { destination, startDate, endDate, groupSize, interests[] }
 - Effect: validates each step before letting you move forward; fires off the request on the last step
- EnrichmentFetcher (server-side, runs in parallel)
 - Input: TripRequest
 - Output: EnrichedContext { nearbyParks: Park[], placeSuggestions: Place[], hotelOptions: Hotel[] }
 - Effect: hits NPS for nearby parks, Google Places for attractions/restaurants, RapidAPI for hotels, all at once. Then bundles the results to feed into Claude as grounding context
- ItineraryGenerator (Claude API call)
 - Input: TripRequest + EnrichedContext
 - Output: a structured Itinerary { days: Day[], summary: string, tips: string[] } as JSON
 - Effect: one Claude call with a system prompt telling it to return only valid JSON; parses and validates before doing anything with it
- CostGuard (wrapper around ItineraryGenerator)
 - Input: user ID + their token usage for the current month from Supabase
 - Output: go or no-go
 - Effect: checks if the user is under their monthly token ceiling before calling Claude; bumps the counter after a successful call; shows a friendly error if they've hit the limit
- TripStore (Supabase DB layer)
 - Input: user ID + the finished Itinerary
 - Output: a tripId

- Effect: writes a row to the trips table so it can be pulled up later on the dashboard
- ResultsPage (/trips/[id])
 - Input: tripId from the URL
 - Output: the full rendered itinerary
 - Effect: fetches the trip from Supabase and renders it as day cards with activities, places, and tips. only visible to the user who made it

After first deliverable goals:

- Trip dashboard: show all your saved trips in one place with destination, dates, and a card layout; let you delete old ones
- Regenerate a trip: tweak your preferences and re-run the AI without having to fill out the whole form again
- Hotel & flight panel: show RapidAPI hotel and flight results right next to the itinerary so you can actually book
- Places enrichment: pull in photos, ratings, and hours from Google Places inline so the itinerary feels real, not just text
- NPS highlights: surface nearby national parks with descriptions and links to the official NPS pages
- Shareable trips: generate a public read-only link that anyone can view without logging in
- Usage UI: let users see how many tokens they've used this month and how much budget they have left